

Abstraction vs Virtualization

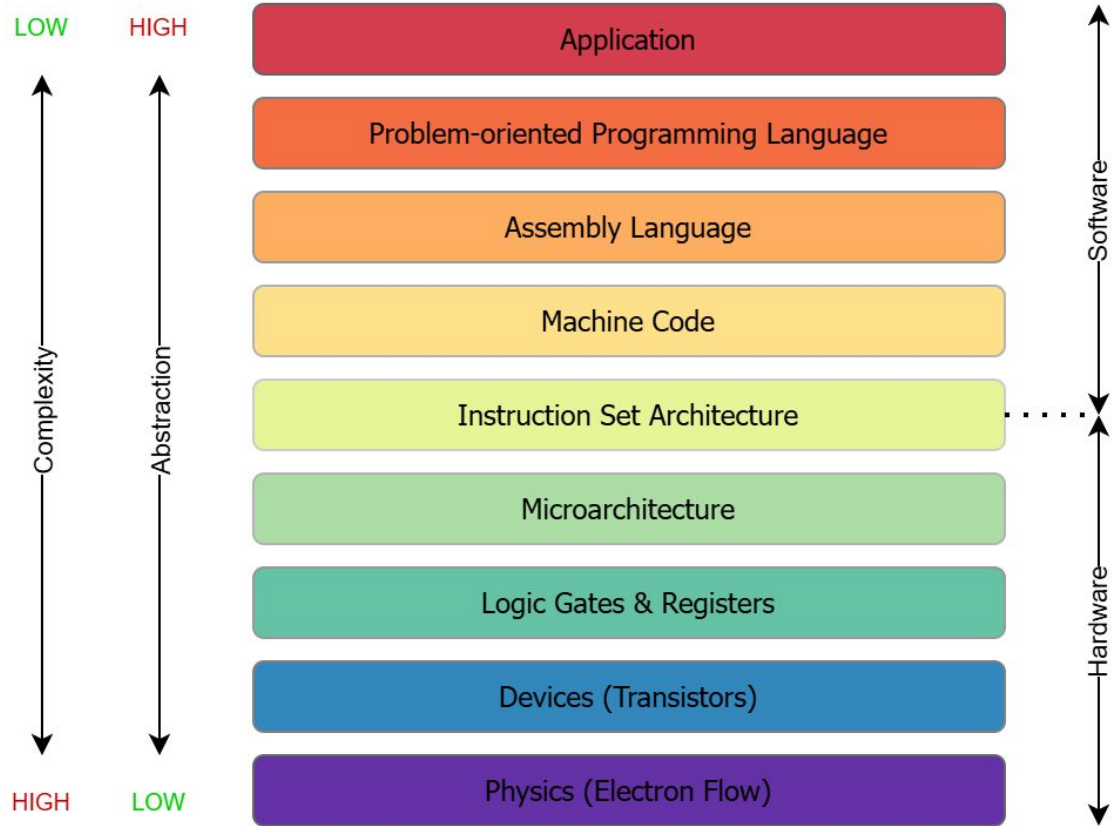
- Abstraction hides complexity. Virtualization share resources
- Virtualization creates the illusion of a resource.
- Virtualization itself is built using abstractions.
- Abstraction simplifies the interaction with resources. Virtualization controls resources or what you appear to own

Abstraction = Complexity

Virtualization = Resources

Abstraction

Layers of Abstraction

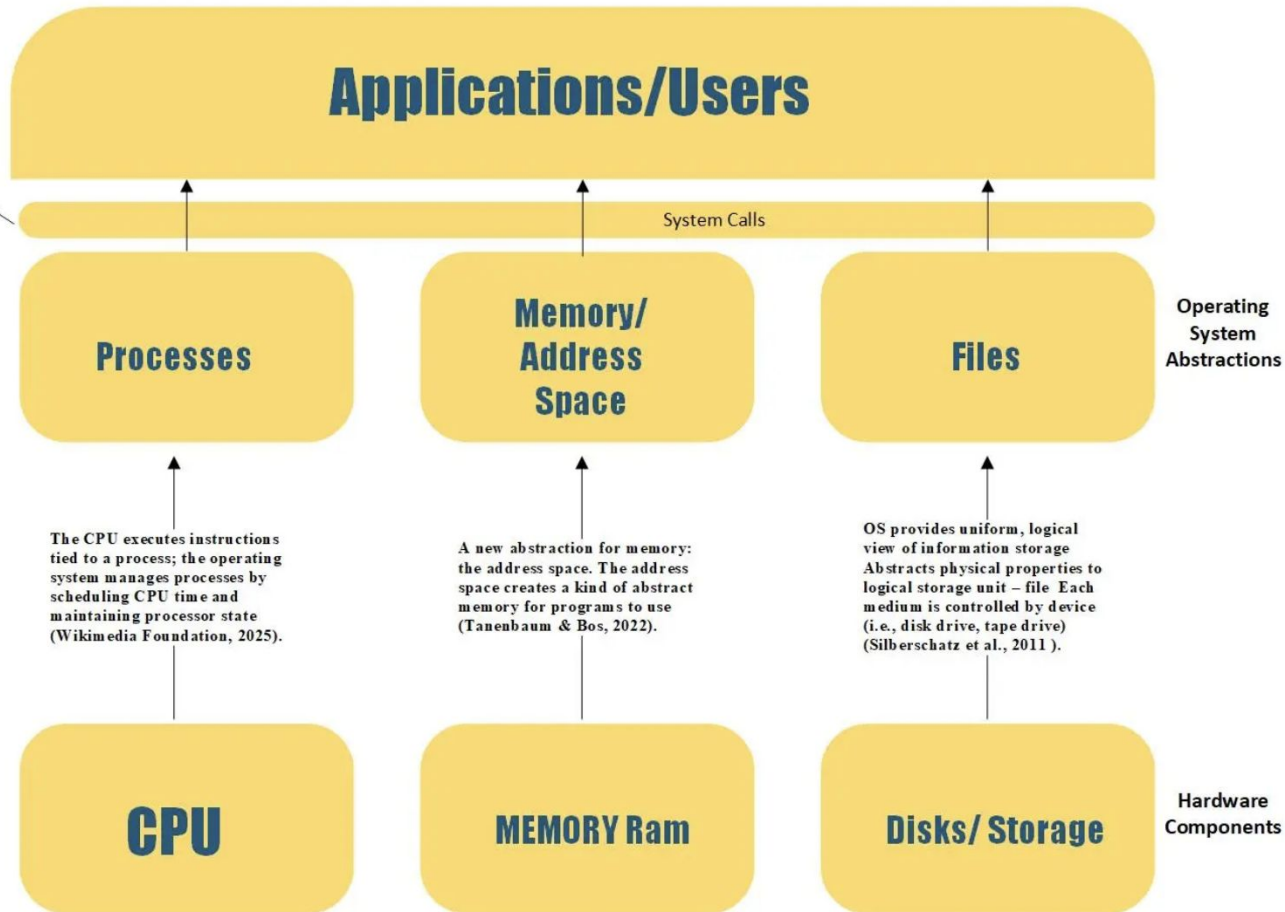


Abstraction

Physical Hardware	What hardware actually provides	OS Abstraction	What the programmer sees
CPU	Executes machine instructions on a limited number of cores	Process / Thread	<code>main()</code> , threads, processes
RAM	Physical memory addresses and memory cells	Virtual Memory / Address Space	Variables, pointers, <code>malloc()</code>
SSD / HDD	Blocks/sectors, controllers, physical storage	File / Directory	<code>open()</code> , <code>read()</code> , <code>write()</code>
Network Card (NIC)	Sends/receives Ethernet/IP packets	Socket	<code>socket()</code> , <code>send()</code> , <code>recv()</code>
Keyboard	Electrical/device events generated by key presses	Input events / Device interface	<code>stdin</code> , keyboard events
Display / GPU	Pixels, framebuffer, GPU commands	Display / graphics API	Window, framebuffer, graphics APIs
Timer / Clock	Hardware timer interrupts/counters	Timer / Time API	<code>sleep()</code> , timers, clocks
USB devices	USB packets, device protocols	Device files / drivers	Read/write device interface

Diagram: The Operating System as an Abstraction

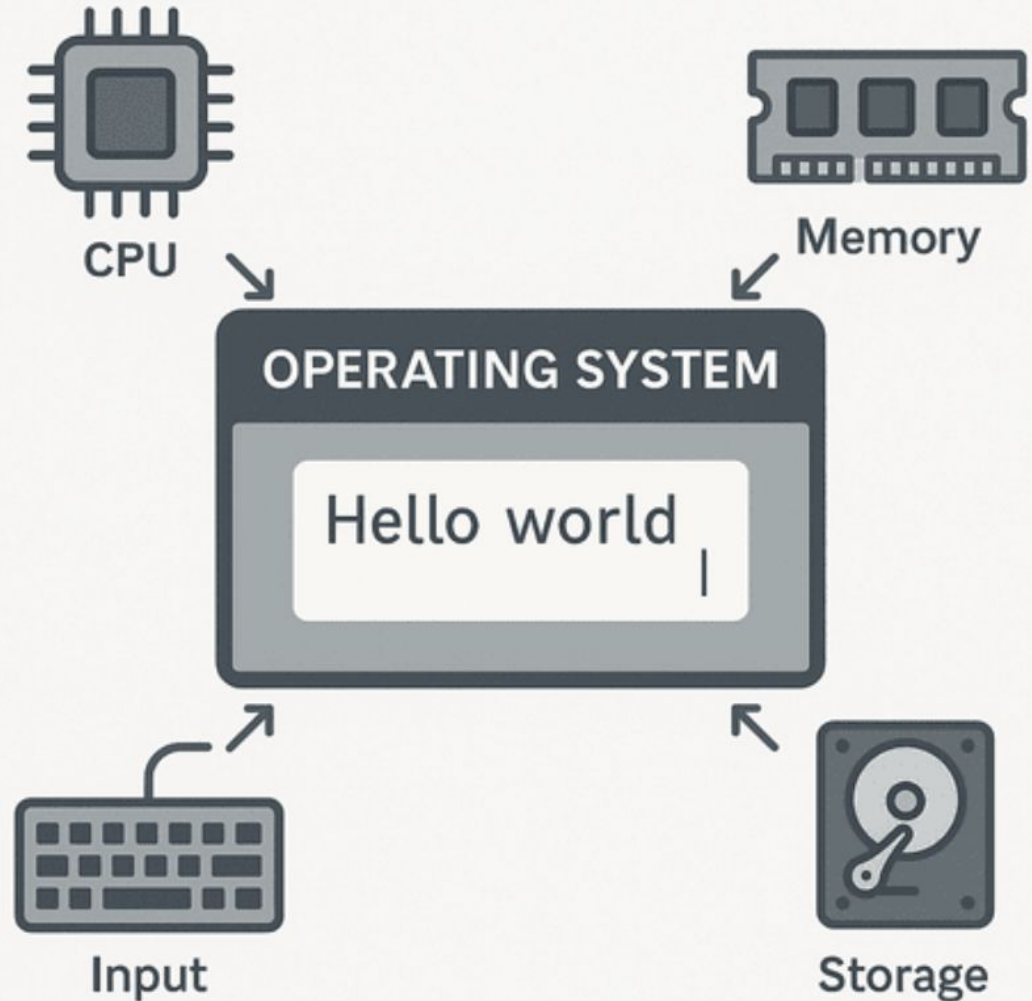
Applications need to access system resources such as memory, files, and networks. System calls provide a standardized interface for applications to request these resources from the OS (Prepbytes, 2023).

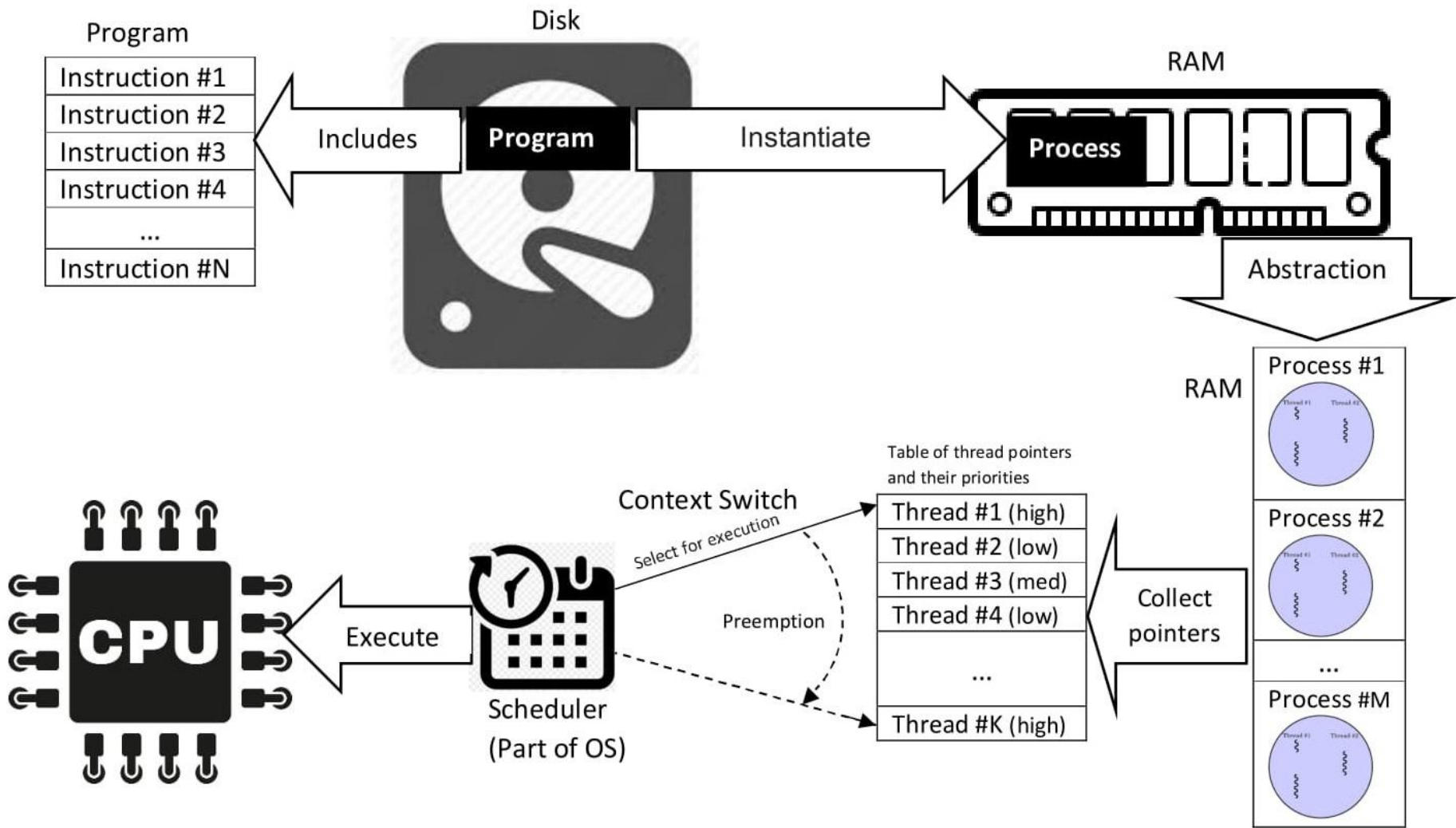


OS: Resource Manager

Resources

- CPU
- Memory or RAM
- I/O: Storage or HDD/SSD, etc





GM-NAA I/O,
produced by
General
Motors for its
IBM 704

1956



Apple II
released

1977



MS-DOS is
released by
Microsoft

1981



Linux is
released
by Linus
Torvalds

1991



Windows 95
is released

1995



android

Android is
released
(based on
a Linux
kernel)

2008



OPENSIFT

OpenShift
released
by Red
Hat

2011

Timeline of Operating Systems

1960s

IBM develops a series of OSs for its 360 series. Multics is developed and abandoned but UNIX is developed as a consequence.

1970s

Unix becomes popular in academic circles and spawns many versions



1980s

The home computer revolution



1990s

Windows dominates the laptop and desktop market



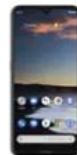
2000s

Unix and then Linux dominate the Supercomputer Market



2010s

Smart phones become ubiquitous after the iPhone release in 2007



C Program Compilation

samp.c

©2004 HowStuffWorks

```
#include <stdio.h>

int main()
{
    printf("Hello!\n");
    return 0;
}
```

samp.c is your C Program. You type samp.c into a text file using a standard text editor. It is human-readable.



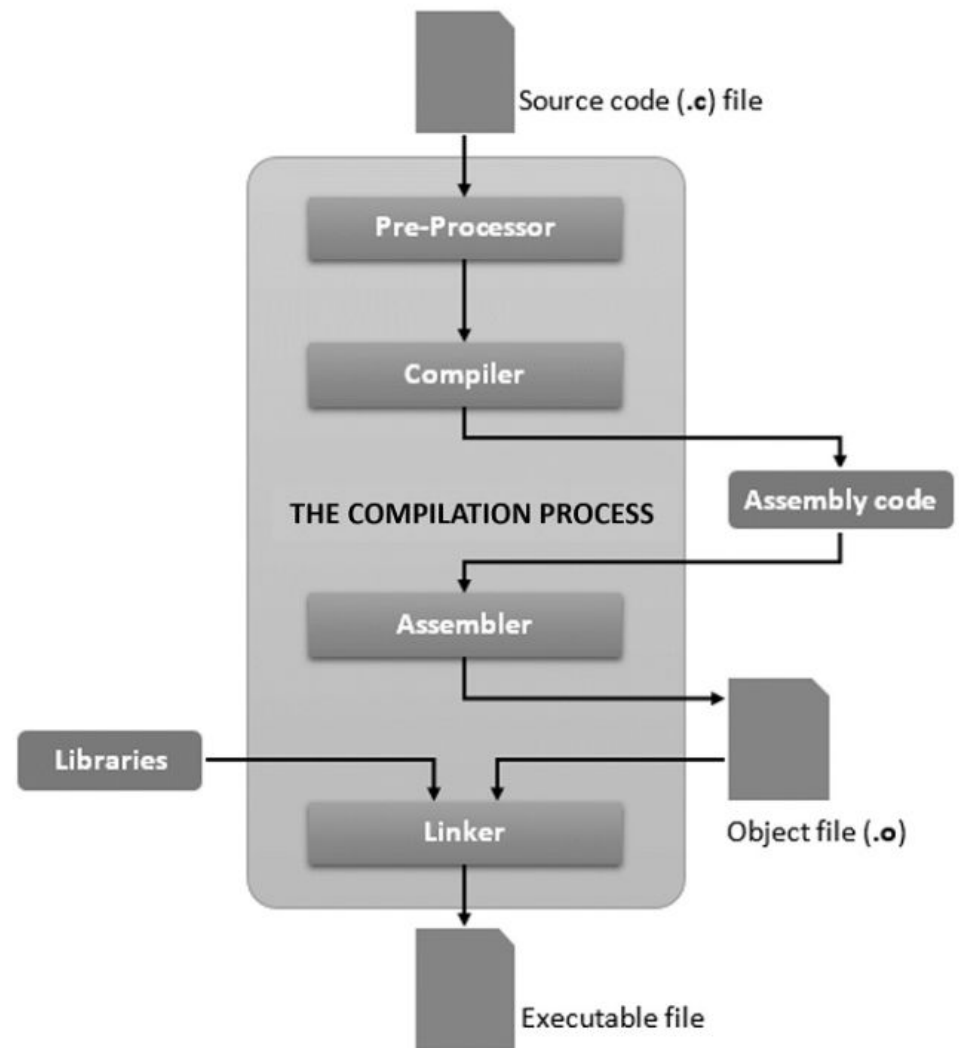
You type:
gcc samp.c -o samp.exe
to compile samp.c into
samp.exe using the gcc
compiler.

samp.exe

```
10110101001011010
10100100100100010
10101001010101110
01010010010110101
11101010100111001
10101001010111110
10101101101001001
10101000111101011
```

The C compiler takes samp.c as input and turns it into a machine-readable executable. The computer "runs" or "executes" this executable.

C Program Compilation



Environment setup

Isolation

- **Container Isolation:** Each container has its own isolated environment, allowing processes to function independently within their own namespace as if they were the only ones running on the system.
 - Each container runs in its own namespace and cannot access or interact with the resources of other containers unless specifically allowed.
 - **Namespace isolation** establishes logical boundaries for each container's processes, networks, and resources to enable the safe and efficient execution of multiple containers on the same system.
-
- Container = isolate not virtualize

Namespace Isolation



Storage

Device Mapper

Btrfs

Aufs

Namespaces

PID

MNT

IPC

UTS

NET

Networking

veth

bridge

iptables

Cgroups

cpu

cpuset

memory

device

Security

Capability

SELinux

seccomp

Namespace

```
vscode → /workspaces/OS $ lsns
```

	NS	TYPE	NPROCS	PID	USER	COMMAND
	4026531834	time	21	117	vscode	/bin/sh
	4026531837	user	21	117	vscode	/bin/sh
	4026532556	mnt	21	117	vscode	/bin/sh
	4026532557	uts	21	117	vscode	/bin/sh
	4026532558	ipc	21	117	vscode	/bin/sh
	4026532559	pid	21	117	vscode	/bin/sh
	4026532560	cgroup	21	117	vscode	/bin/sh
	4026532561	net	21	117	vscode	/bin/sh

Userspace vs Namespace

- Userspace is where your application lives. A **memory area** where normal applications, user programs, and shells run.
- Namespace is a kernel mechanism that controls the application's view of the system. It **isolates global system resources** for a group of processes.
- Userspace **protects the operating system** core from crashing via bad app code.
- Namespaces **hide system resources** from other processes to create containers
- Userspace: **devcontainers/base:noble**

Docker VM Managers

Docker Desktop for Mac currently supports different VM managers:

- Docker VMM — Apple Silicon
- Apple Virtualization framework
- QEMU — legacy on Apple Silicon
- HyperKit — legacy on Intel Macs

Summary

1. VM

Virtualizes hardware

Provides a Linux kernel

Example: Docker Desktop's Linux VM

2. Container

Isolates processes

Shares the Linux kernel

Uses namespaces + cgroups + capabilities

3. Userspace

Where applications execute

bash, gcc, Node, your C program, libc, etc.

